

Advanced Locks in Java

The `synchronized` keyword is often enough for protecting a critical section. It automatically acquires an intrinsic lock when a thread enters the block or method and releases it when the thread exits.

Java also provides explicit locking utilities in `java.util.concurrent.locks`. These APIs are useful when an application needs more control over lock acquisition, interruption, timeouts, fairness, multiple waiting conditions, or read-heavy access patterns.

This chapter covers:

- the `Lock` interface
- `ReentrantLock`
- `ReadWriteLock` and `ReentrantReadWriteLock`
- `StampedLock`
- `Semaphore`
- `Condition`

1. Why Go Beyond `synchronized` ?

A synchronized block is simple and safe:

```
synchronized (lockObject) {  
    // critical section  
}
```

The JVM handles lock release automatically, even if the block throws an exception.

However, intrinsic locks do not directly provide some forms of control.

1.1 No Immediate Try-and-Fallback Operation

With `synchronized`, a thread waits until the monitor becomes available.

There is no direct equivalent of:

```
Try once.  
If the lock is unavailable, perform fallback work.
```

Explicit locks provide `tryLock()` for this purpose.

1.2 No Timed Lock Acquisition

A synchronized block cannot directly express:

```
Wait for the lock for at most two seconds.  
If it is still unavailable, stop waiting.
```

`Lock.tryLock(timeout, unit)` supports timed acquisition.

1.3 Lock Acquisition Is Not Interruptible

A thread waiting to enter a synchronized block cannot use a dedicated interruptible lock-acquisition method.

The `Lock` interface provides:

```
lock.lockInterruptibly();
```

This allows a waiting thread to respond to interruption.

1.4 No Configurable Fairness

Intrinsic locks do not expose a fairness policy.

Some explicit-lock implementations can be created in fair mode, which generally favours threads that have waited longer.

Fairness can improve predictability but usually reduces throughput.

1.5 Only One Monitor Wait Set

With intrinsic locking, `wait()`, `notify()` and `notifyAll()` operate on the monitor's single wait set.

A `Lock` can create multiple `Condition` objects:

```
Condition notEmpty = lock.newCondition();
Condition notFull = lock.newCondition();
```

This allows different groups of threads to wait for different state predicates.

2. The `Lock` Interface

The `Lock` interface belongs to:

```
java.util.concurrent.locks.Lock
```

It separates lock acquisition and release into explicit method calls.

Important methods include:

```
void lock();
void lockInterruptibly() throws InterruptedException;
boolean tryLock();
boolean tryLock(long time, TimeUnit unit)
    throws InterruptedException;
void unlock();
Condition newCondition();
```

A `Lock` implementation provides synchronization semantics similar to monitor locking, but with additional capabilities.

3. Basic Locking Pattern

```
Lock lock = new ReentrantLock();

lock.lock();

try {
    // critical section
} finally {
    lock.unlock();
}
```

The `finally` block is essential.

If an exception occurs and `unlock()` is skipped, the lock can remain held indefinitely.

Acquire the lock before entering `try`, and release it in `finally`.

Do not place `lock.lock()` inside the `try` block unless the code is carefully structured. If acquisition itself fails before the lock is obtained, `finally` must not call `unlock()`.

4. ReentrantLock

`ReentrantLock` is the most commonly used implementation of the `Lock` interface.

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

Lock lock = new ReentrantLock();
```

It provides:

- mutual exclusion
- reentrancy
- immediate lock attempts

- timed lock attempts
- interruptible acquisition
- optional fairness
- condition variables
- diagnostic methods

5. What Does Reentrant Mean?

A reentrant lock allows the thread that already owns the lock to acquire it again without blocking itself.

```
lock.lock(); // hold count = 1
lock.lock(); // hold count = 2
```

The lock maintains conceptually:

```
Owner thread
+
Hold count
```

Each successful acquisition by the owner increments the hold count.

Each `unlock()` decrements it.

```
lock.lock(); // count = 1
lock.lock(); // count = 2

lock.unlock(); // count = 1
lock.unlock(); // count = 0, lock released
```

The lock becomes available to another thread only when the hold count reaches zero.

6. Why Reentrancy Is Necessary

Consider one lock-protected method calling another method that uses the same lock.

```
class BankAccount {

    private int balance;
    private final ReentrantLock lock =
        new ReentrantLock();

    public void deposit(int amount) {
        lock.lock();

        try {
            validateAmount(amount);
            balance += amount;
        } finally {
            lock.unlock();
        }
    }

    private void validateAmount(int amount) {
        lock.lock();

        try {
            if (amount <= 0) {
                throw new IllegalArgumentException(
                    "Amount must be positive"
                );
            }
        } finally {
            lock.unlock();
        }
    }
}
```

```
}  
}
```

`deposit()` acquires the lock and then calls `validateAmount()`, which acquires the same lock again.

Because the lock is reentrant, the thread does not deadlock with itself.

In real code, a private validation method may not need to acquire the lock independently. The example demonstrates that independently synchronized methods can safely call one another when they use the same reentrant lock.

7. `synchronized` Is Also Reentrant

Intrinsic monitors are reentrant too.

```
public synchronized void methodA() {  
    methodB();  
}  
  
public synchronized void methodB() {  
    // same thread can enter  
}
```

`ReentrantLock` is not useful merely because it is reentrant. Its main advantage is the additional acquisition, fairness and condition-management APIs.

Important `ReentrantLock` Methods

8. `lock()`

```
lock.lock();
```

If the lock is available, the current thread acquires it.

If another thread owns it, the current thread waits.

Example:

```
class Counter {

    private int count;
    private final ReentrantLock lock =
        new ReentrantLock();

    public void increment() {
        lock.lock();

        try {
            count++;
        } finally {
            lock.unlock();
        }
    }

    public int getCount() {
        lock.lock();

        try {
            return count;
        } finally {
            lock.unlock();
        }
    }
}
```

9. unlock()

```
lock.unlock();
```


Only the thread that currently owns a `ReentrantLock` may release it.

Calling `unlock()` without owning the lock throws:

`IllegalMonitorStateException`

Every successful acquisition must have a matching release.

```
lock.lock();
lock.lock();

lock.unlock(); // still held
lock.unlock(); // fully released
```

Avoid defensive patterns that silently unlock whenever `isHeldByCurrentThread()` returns true. Balanced acquisition and release should be clear from the code structure.

10. `tryLock()`

```
boolean acquired = lock.tryLock();
```

`tryLock()` attempts to acquire the lock immediately.

It returns:

```
true  → lock acquired
false → lock unavailable
```

Example:

```
if (lock.tryLock()) {
    try {
        performCriticalWork();
    } finally {
```

```
        lock.unlock();
    }
} else {
    performFallbackWork();
}
```

This is useful when waiting would be more expensive than using an alternative path.

11. Timed `tryLock()`

```
boolean acquired =
    lock.tryLock(
        2,
        TimeUnit.SECONDS
    );
```

The thread waits for up to the requested time.

```
try {
    if (
        lock.tryLock(
            2,
            TimeUnit.SECONDS
        )
    ) {
        try {
            performCriticalWork();
        } finally {
            lock.unlock();
        }
    } else {
        performFallbackWork();
    }
}
```

```
} catch (InterruptedException exception) {  
    Thread.currentThread().interrupt();  
}
```

Possible outcomes:

Lock becomes available in time
→ acquire and return true

Timeout expires
→ return false

Waiting thread is interrupted
→ throw InterruptedException

12. `lockInterruptibly()`

```
try {  
    lock.lockInterruptibly();  
  
    try {  
        performCriticalWork();  
    } finally {  
        lock.unlock();  
    }  
} catch (InterruptedException exception) {  
    Thread.currentThread().interrupt();  
}
```

Unlike `lock()`, this method allows a thread waiting for the lock to respond to interruption.

It is useful for cancellable tasks and shutdown-sensitive systems.

13. `isLocked()`

```
boolean locked = lock.isLocked();
```

This reports whether any thread currently holds the lock.

It is mainly useful for:

- monitoring
- testing
- diagnostics

Do not use it for check-then-act business logic:

```
if (!lock.isLocked()) {
    // lock may become occupied before this code runs
}
```

The observed state can change immediately.

Use `tryLock()` when the action depends on actually acquiring the lock.

14. `isHeldByCurrentThread()`

```
boolean owned =
    lock.isHeldByCurrentThread();
```

This checks whether the current thread owns the lock.

It is useful for:

- assertions
- diagnostic helper methods
- validating internal lock requirements

Example:

```
private void verifyLockOwnership() {
    if (!lock.isHeldByCurrentThread()) {
```

```
        throw new IllegalStateException(  
            "Current thread must hold the lock"  
        );  
    }  
}
```

15. `getHoldCount()`

```
int count = lock.getHoldCount();
```

This returns the number of times the current thread has acquired the lock without matching releases.

For a thread that does not own the lock, it returns zero.

It is primarily a diagnostic method.

16. Fair and Non-Fair Locks

A non-fair lock is the default:

```
ReentrantLock lock =  
    new ReentrantLock();
```

A fair lock is created with:

```
ReentrantLock lock =  
    new ReentrantLock(true);
```

Non-Fair Lock

A newly arriving thread may acquire the lock before threads that have waited longer.

This behaviour is called **barging**.

Non-fair locks usually provide better throughput because a running thread may acquire the lock without additional queue-management overhead.

Fair Lock

A fair lock generally favours the longest-waiting thread under contention.

Approximately arrival-order acquisition

Fairness does not mean mathematically strict scheduling order. Thread scheduling and untimed `tryLock()` can still affect the observed order.

Use fair locks only when starvation risk or predictable access order matters more than maximum throughput.

17. Queue Diagnostic Methods

`ReentrantLock` also exposes methods such as:

```
lock.hasQueuedThreads();  
lock.getQueueLength();  
lock.hasQueuedThread(thread);
```

They provide estimates or snapshots useful for monitoring.

Do not build correctness rules around them because queue state can change immediately after inspection.

18. `synchronized` vs `ReentrantLock`

Capability	<code>synchronized</code>	<code>ReentrantLock</code>
Automatic release	Yes	No
Reentrant	Yes	Yes
Immediate try	No	Yes
Timed acquisition	No	Yes

Capability	<code>synchronized</code>	<code>ReentrantLock</code>
Interruptible acquisition	No dedicated API	Yes
Configurable fairness	No	Yes
Multiple condition queues	No	Yes
Simpler syntax	Yes	No
Diagnostic APIs	Limited	More extensive

Use `synchronized` when its simpler structure is sufficient.

Use `ReentrantLock` when a specific explicit-lock feature is required.

Part 2: Read-Write Locks

19. The Problem With One Exclusive Lock

A normal exclusive lock allows only one thread inside the protected region.

```
Reader 1 enters
Reader 2 waits
Reader 3 waits
```

This is necessary for writes, but it can be unnecessarily restrictive for read-only access.

If several threads only inspect stable data, they may be able to proceed together.

20. Read and Write Operations

Shared-resource operations can often be divided into:

Reads

- observe data
- do not change the protected state

- can potentially run concurrently

Writes

- modify protected state
- must not overlap with readers or other writers

Core rule:

Multiple readers	→ allowed
One writer	→ allowed alone
Reader with writer	→ not allowed
Multiple writers	→ not allowed

21. The **ReadWriteLock** Interface

ReadWriteLock provides two related locks:

```
Lock readLock();
Lock writeLock();
```

The standard implementation is:

```
ReadWriteLock lock =
    new ReentrantReadWriteLock();
```

Obtain the individual locks:

```
Lock readLock = lock.readLock();
Lock writeLock = lock.writeLock();
```

22. Basic **ReentrantReadWriteLock** Example


```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;

class SharedData {

    private int value;

    private final ReadWriteLock lock =
        new ReentrantReadWriteLock();

    private final Lock readLock =
        lock.readLock();

    private final Lock writeLock =
        lock.writeLock();

    public int read() {
        readLock.lock();

        try {
            return value;
        } finally {
            readLock.unlock();
        }
    }

    public void write(int newValue) {
        writeLock.lock();

        try {
            value = newValue;
        } finally {
            writeLock.unlock();
        }
    }
}
```

```
}  
}
```

Several threads can execute `read()` together.

A thread executing `write()` receives exclusive access.

23. Why Readers Need a Lock

Read locks are not mainly used to protect readers from other readers.

They coordinate readers with writers.

Without a read lock:

```
Reader begins reading several related fields  
    ↓  
Writer modifies those fields midway  
    ↓  
Reader may observe an inconsistent combination
```

A read lock prevents a writer from modifying the protected state until the reader finishes.

24. When Read-Write Locks Help

A read-write lock may improve concurrency when:

- reads are much more frequent than writes
- read operations take enough time for concurrency to matter
- the protected state is shared by many threads
- writes must remain exclusive

Examples:

- in-memory caches
- configuration snapshots

- lookup tables
- read-heavy registries
- shared indexes

It may not help when:

- critical sections are extremely short
- writes are frequent
- contention is low
- lock-management overhead exceeds the benefit

Measure rather than assuming that a read-write lock is faster.

25. Fairness and Starvation

Create a fair read-write lock with:

```
ReentrantReadWriteLock lock =
    new ReentrantReadWriteLock(true);
```

The default is non-fair:

```
ReentrantReadWriteLock lock =
    new ReentrantReadWriteLock();
```

A continuous flow of readers can delay a waiting writer in some access patterns.

Fair mode generally grants access according to waiting order and can reduce starvation risk, but it usually lowers throughput.

Fairness is a policy trade-off, not an absolute guarantee about operating-system scheduling.

26. Reentrancy

Both the read lock and write lock are reentrant.

```
readLock.lock();  
readLock.lock();  
  
readLock.unlock();  
readLock.unlock();
```

The write-lock owner can reacquire the write lock.

A thread holding the write lock may also acquire the read lock. This enables lock downgrading.

27. Lock Downgrading

Downgrading means changing:

```
write lock → read lock
```

The safe sequence is:

1. acquire the write lock
2. update the state
3. acquire the read lock while still holding the write lock
4. release the write lock
5. continue reading
6. release the read lock

Example:

```
writeLock.lock();  
  
try {  
    value = 100;  
  
    readLock.lock();
```

```

} finally {
    writeLock.unlock();
}

try {
    System.out.println(value);
} finally {
    readLock.unlock();
}

```

Acquiring the read lock before releasing the write lock prevents another writer from modifying the state in between.

28. Lock Upgrading

Upgrading means changing:

read lock → write lock

A direct upgrade is not supported safely.

```

readLock.lock();

try {
    writeLock.lock(); // deadlock risk
} finally {
    readLock.unlock();
}

```

Several threads may all hold read locks and wait for the write lock. The write lock cannot be granted until all read locks are released.

The usual approach is:

1. release the read lock

2. acquire the write lock
3. recheck the state
4. perform the update

The state must be rechecked because another thread may have changed it during the transition.

Part 3: `StampedLock`

29. What Is `StampedLock` ?

`StampedLock` is an advanced synchronization utility designed for read-heavy workloads.

It supports three main modes:

```
writeLock()  
readLock()  
tryOptimisticRead()
```

Its distinctive feature is **optimistic reading**.

```
import java.util.concurrent.locks.StampedLock;
```

30. What Is a Stamp?

Lock operations return a `long` stamp:

```
long stamp = lock.readLock();
```

The stamp is a token representing that particular lock mode and acquisition state.

It must be supplied when releasing or converting the lock:

```
lock.unlockRead(stamp);
```

A stamp should not be treated as a simple application version number or thread ID.

31. Pessimistic Read Lock

```
long stamp = lock.readLock();

try {
    return value;
} finally {
    lock.unlockRead(stamp);
}
```

Several read-lock holders may proceed together.

A writer cannot acquire the write lock until pessimistic readers release their stamps.

32. Write Lock

```
long stamp = lock.writeLock();

try {
    value = newValue;
} finally {
    lock.unlockWrite(stamp);
}
```

The write lock is exclusive.

No read or write lock can coexist with it.

33. Optimistic Read

An optimistic read does not acquire a traditional read lock.

```
long stamp = lock.tryOptimisticRead();
```

The reader:

1. obtains an optimistic stamp
2. copies the required fields into local variables
3. validates the stamp
4. falls back to a read lock if validation fails

```
Assume no writer interferes
    ↓
Read local snapshot
    ↓
Validate
├─ valid   → use snapshot
└─ invalid → reread under read lock
```

34. Optimistic Read Example

```
import java.util.concurrent.locks.StampedLock;

class SharedData {

    private int value;
    private final StampedLock lock =
        new StampedLock();

    public void write(int newValue) {
        long stamp = lock.writeLock();

        try {
```



```
        value = newValue;
    } finally {
        lock.unlockWrite(stamp);
    }
}

public int read() {
    long stamp =
        lock.tryOptimisticRead();

    int current = value;

    if (!lock.validate(stamp)) {
        stamp = lock.readLock();

        try {
            current = value;
        } finally {
            lock.unlockRead(stamp);
        }
    }

    return current;
}
```

`validate(stamp)` reports whether a write-lock acquisition invalidated the optimistic observation.

35. Reading Multiple Fields Optimistically

The purpose of validation becomes clearer when several related fields form one state.

```
class Point {

    private double x;
    private double y;

    private final StampedLock lock =
        new StampedLock();

    public double distanceFromOrigin() {
        long stamp =
            lock.tryOptimisticRead();

        double currentX = x;
        double currentY = y;

        if (!lock.validate(stamp)) {
            stamp = lock.readLock();

            try {
                currentX = x;
                currentY = y;
            } finally {
                lock.unlockRead(stamp);
            }
        }

        return Math.sqrt(
            currentX * currentX +
            currentY * currentY
        );
    }
}
```

Always copy the shared fields before validation.

Do not repeatedly read the shared fields after validation, because a writer may change them later.

36. When Optimistic Reads Help

Optimistic reads are useful when:

- reads are extremely frequent
- writes are rare
- reading the state is quick
- retrying under a real read lock is inexpensive
- profiling shows ordinary read locking is a bottleneck

They are less useful when:

- writes are frequent
 - the read operation is long
 - the object graph is difficult to copy consistently
 - implementation simplicity matters more than marginal throughput
-

37. **StampedLock** Is Not Reentrant

This is a major difference from **ReentrantLock**.

```
long firstStamp = lock.writeLock();
long secondStamp = lock.writeLock(); // can deadlock
```

The lock does not recognise the second acquisition as belonging to an existing owner in the same way a reentrant lock does.

Methods using the same **StampedLock** must be designed carefully to avoid nested reacquisition.

38. No Traditional Thread Ownership

`StampedLock` primarily manages lock state through stamps rather than monitor-style ownership checks.

The stamp returned by acquisition must be preserved and used correctly.

```
long stamp = lock.readLock();

try {
    // read
} finally {
    lock.unlockRead(stamp);
}
```

Passing an invalid stamp can result in:

`IllegalMonitorStateException`

Do not transfer stamps casually between unrelated operations.

39. Stamp Conversion

`StampedLock` can attempt to convert a held mode without fully releasing it.

Important methods include:

```
tryConvertToWriteLock(stamp)
tryConvertToReadLock(stamp)
tryConvertToOptimisticRead(stamp)
```

Example:

```
long stamp = lock.readLock();

try {
    while (value == 0) {
```

```

        long writeStamp =
            lock.tryConvertToWriteLock(
                stamp
            );

        if (writeStamp != 0L) {
            stamp = writeStamp;
            value = 1;
            break;
        }

        lock.unlockRead(stamp);
        stamp = lock.writeLock();
    }
} finally {
    lock.unlock(stamp);
}

```

Conversions are advanced APIs and require careful tracking of the current stamp mode.

40. ReadWriteLock vs StampedLock

Feature	ReentrantReadWriteLock	StampedLock
Multiple readers	Yes	Yes
Exclusive writer	Yes	Yes
Optimistic reads	No	Yes
Reentrant	Yes	No
Condition support	Write lock supports conditions	No condition support
Lock ownership model	Thread-oriented	Stamp-oriented
Simpler to use	Yes	No

Feature	<code>ReentrantReadWriteLock</code>	<code>StampedLock</code>
Best fit	General read-heavy protection	Specialised, measured read-heavy optimisation

Prefer `ReentrantReadWriteLock` unless optimistic reads provide a demonstrated benefit.

Part 4: Semaphore

41. What Is a Semaphore?

A `Semaphore` maintains a number of permits.

```
Semaphore semaphore =
    new Semaphore(3);
```

A thread must acquire a permit before entering the limited region and release it afterward.

```
Available permit
    → thread proceeds

No available permit
    → thread waits
```

With three permits, at most three successful acquirers can proceed concurrently.

42. Basic Semaphore Pattern

```
Semaphore semaphore =
    new Semaphore(3);

boolean acquired = false;
```

```
try {
    semaphore.acquire();
    acquired = true;

    useLimitedResource();

} catch (InterruptedException exception) {
    Thread.currentThread().interrupt();

} finally {
    if (acquired) {
        semaphore.release();
    }
}
```

Release a permit only if acquisition succeeded.

Otherwise, the permit count can be increased incorrectly.

43. `acquire()`

```
semaphore.acquire();
```

This acquires one permit.

If none are available, the thread waits.

The operation is interruptible and may throw:

```
InterruptedException
```

Multiple permits can also be requested:

```
semaphore.acquire(3);
```

44. `release()`

```
semaphore.release();
```

This returns one permit.

Multiple permits can be returned:

```
semaphore.release(3);
```

A semaphore does not track ownership in the same way as a lock.

Any thread can call `release()`, and an incorrect extra release can increase the permit count beyond its intended level.

Balanced permit management is the application's responsibility.

45. Binary Semaphore

```
Semaphore semaphore =  
    new Semaphore(1);
```

Only one permit exists, so at most one successful acquirer proceeds at a time.

This resembles mutual exclusion, but it is not identical to a lock:

- no owner thread is tracked
- another thread may release the permit
- reentrancy is not provided
- extra releases can increase permits

Use a `Lock` for ownership-based mutual exclusion.

Use a semaphore when the permit model is the actual requirement.

46. Counting Semaphore


```
Semaphore semaphore =
    new Semaphore(5);
```

Up to five permits can be held simultaneously.

This is useful for limiting access to a finite-capacity resource.

Examples:

- database connections
- external-service concurrency
- file-processing slots
- limited hardware devices
- bounded parallel tasks

A semaphore limits **concurrent access**. It is not by itself a requests-per-second rate limiter.

47. Timed and Immediate Permit Acquisition

Immediate attempt:

```
boolean acquired =
    semaphore.tryAcquire();
```

Timed attempt:

```
boolean acquired =
    semaphore.tryAcquire(
        2,
        TimeUnit.SECONDS
    );
```

These methods allow fallback logic when capacity is unavailable.

48. Fair Semaphore

Default non-fair semaphore:

```
Semaphore semaphore =  
    new Semaphore(3);
```

Fair semaphore:

```
Semaphore semaphore =  
    new Semaphore(3, true);
```

Fair mode generally serves queued blocking acquirers in arrival order.

As with fair locks, scheduling is not a perfect strict FIFO guarantee, and untimed `tryAcquire()` may barge.

Fairness usually reduces throughput.

49. Resource-Limiting Example

```
import java.util.concurrent.Semaphore;  
  
class ExternalServiceClient {  
  
    private final Semaphore permits =  
        new Semaphore(10);  
  
    public String callService()  
        throws InterruptedException {  
  
        permits.acquire();  
  
        try {  
            return performRequest();  
        } finally {
```

```

        permits.release();
    }
}

private String performRequest() {
    return "response";
}
}

```

At most ten calls execute inside `performRequest()` concurrently.

The number of application threads can be much larger than the number of permits.

Part 5: Condition Variables

50. What Is a **Condition** ?

A **Condition** is a waiting mechanism associated with a **Lock**.

```

ReentrantLock lock =
    new ReentrantLock();

Condition condition =
    lock.newCondition();

```

It plays a role similar to:

```

Object.wait()
Object.notify()
Object.notifyAll()

```

The corresponding Condition methods are:

```
wait()      → await()
notify()    → signal()
notifyAll() → signalAll()
```

51. Why Conditions Exist

A lock protects a state transition.

A condition allows a thread to wait until the protected state satisfies a particular predicate.

Example predicates:

```
Queue is not empty
Queue is not full
Account has enough balance
Resource has become available
Task has completed
```

The lock and condition work together.

52. `await()`

```
condition.await();
```

The calling thread must hold the associated lock.

Conceptually, `await()`:

1. atomically releases the lock
2. places the thread in the condition wait set
3. suspends the thread
4. later reacquires the lock before returning

```

Hold lock
  ↓
Condition not satisfied
  ↓
await()
  ↓
Release lock and wait
  ↓
Receive signal
  ↓
Compete to reacquire lock
  ↓
Return only after lock is reacquired

```

`await()` can throw `InterruptedException`.

53. `signal()`

```
condition.signal();
```

The current thread must hold the associated lock.

`signal()` selects one waiting thread, if one exists.

The selected thread does not immediately run inside the critical section. It must first reacquire the lock after the signalling thread releases it.

54. `signalAll()`

```
condition.signalAll();
```

This signals all threads waiting on that condition.

They then compete to reacquire the lock one at a time.

Use `signalAll()` when:

- several different predicates may depend on the same condition
- it is difficult to know which waiter can proceed

- missing a relevant waiter would risk indefinite waiting

Use `signal()` when one waiter is enough and the waiting design is well understood.

55. Lock Ownership Requirement

The current thread must hold the lock before calling:

```
condition.await();  
condition.signal();  
condition.signalAll();
```

Otherwise, the method throws:

```
IllegalMonitorStateException
```

Always use a condition inside the lock's critical section.

56. Always Wait in a `while` Loop

Incorrect:

```
if (queue.isEmpty()) {  
    notEmpty.await();  
}
```

Correct:

```
while (queue.isEmpty()) {  
    notEmpty.await();  
}
```

A waiting thread must recheck the predicate because:

- spurious wakeups are permitted

- another thread may consume the state before this thread reacquires the lock
- the signal indicates that state may have changed, not that the predicate is guaranteed to be true

The condition queue is a notification mechanism. The shared-state predicate is the source of truth.

57. Multiple Condition Queues

A major advantage over intrinsic monitor methods is that one lock can have several conditions.

```
Condition notEmpty =  
    lock.newCondition();  
  
Condition notFull =  
    lock.newCondition();
```

Producers wait only on `notFull`.

Consumers wait only on `notEmpty`.

This avoids waking unrelated threads unnecessarily.

58. Bounded Buffer With Conditions

```
import java.util.ArrayDeque;  
import java.util.Queue;  
import java.util.concurrent.locks.Condition;  
import java.util.concurrent.locks.ReentrantLock;  
  
class BoundedBuffer<T> {  
  
    private final Queue<T> queue =  
        new ArrayDeque<>();
```

```

private final int capacity;

private final ReentrantLock lock =
    new ReentrantLock();

private final Condition notEmpty =
    lock.newCondition();

private final Condition notFull =
    lock.newCondition();

BoundedBuffer(int capacity) {
    if (capacity <= 0) {
        throw new IllegalArgumentException(
            "Capacity must be positive"
        );
    }

    this.capacity = capacity;
}

public void put(T item)
    throws InterruptedException {

    lock.lockInterruptibly();

    try {
        while (queue.size() == capacity) {
            notFull.await();
        }

        queue.add(item);

        notEmpty.signal();
    } finally {

```



```

        lock.unlock();
    }
}

public T take()
    throws InterruptedException {

    lock.lockInterruptibly();

    try {
        while (queue.isEmpty()) {
            notEmpty.await();
        }

        T item = queue.remove();

        notFull.signal();

        return item;
    } finally {
        lock.unlock();
    }
}
}

```

Flow:

```

Producer:
queue full
    → await notFull

Consumer removes item
    → signal notFull

Consumer:
queue empty
    → await notEmpty

```

Producer adds item
→ signal notEmpty

In production code, prefer an existing `BlockingQueue` unless implementing the mechanism for learning or requiring custom behaviour.

59. Timed Condition Waiting

A condition also provides timed waiting methods.

```
condition.await(  
    2,  
    TimeUnit.SECONDS  
);
```

Other variants include:

```
awaitNanos(...)  
awaitUntil(...)  
awaitUninterruptibly()
```

Timed waiting is useful when indefinite blocking is unacceptable.

`awaitUninterruptibly()` should be used carefully because it delays the thread's response to cancellation.

60. Condition vs `wait()` and `notify()`

Capability	Monitor methods	Condition
Associated with	Intrinsic monitor	Explicit <code>Lock</code>
Wait operation	<code>wait()</code>	<code>await()</code>
Wake one	<code>notify()</code>	<code>signal()</code>
Wake all	<code>notifyAll()</code>	<code>signalAll()</code>
Releases lock while waiting	Yes	Yes

Capability	Monitor methods	Condition
Reacquires before returning	Yes	Yes
Multiple wait sets per lock	No	Yes
Timed variants	Yes	Yes
Interruptible waiting	Yes	Yes

Use the mechanism that matches the chosen locking model.

Do not mix a `Condition` with a different lock or call monitor methods as substitutes inside a `ReentrantLock` design.

Part 6: Choosing the Correct Tool

61. Use `synchronized` When

- mutual exclusion is straightforward
- no timed or interruptible acquisition is needed
- one wait set is sufficient
- automatic lock release is valuable
- simpler code is preferable

For many application-level critical sections, `synchronized` remains the best default.

62. Use `ReentrantLock` When

- `tryLock()` is required
- timed acquisition is required
- lock acquisition must be interruptible
- optional fairness is required
- multiple condition queues are needed
- lock diagnostic methods are useful

63. Use **ReentrantReadWriteLock** When

- the workload is read-heavy
 - reads can safely overlap
 - writes must be exclusive
 - optimistic reads are unnecessary
 - reentrancy is needed
-

64. Use **StampedLock** When

- the workload is strongly read-dominated
- writes are rare
- optimistic reads can be validated cheaply
- profiling justifies additional complexity
- reentrancy and conditions are not required

Do not choose it merely because it is newer.

65. Use **Semaphore** When

- the goal is to limit concurrent access
- several threads may enter simultaneously
- access is represented by permits
- ownership-based mutual exclusion is not required

Examples include connection limits, external-service concurrency and fixed-capacity resource access.

66. Use **Condition** When

- a thread must wait for a state predicate

- the state is protected by a `Lock`
- separate groups of waiters need separate queues
- producer-consumer-style coordination is required

Use higher-level utilities such as `BlockingQueue`, `CountDownLatch` or `Semaphore` when they directly match the problem.

Quick Comparison

Tool	Main purpose	Key characteristic
<code>synchronized</code>	Basic mutual exclusion	Automatic acquisition and release
<code>ReentrantLock</code>	Flexible exclusive locking	Try, timeout, interruption and fairness
<code>ReentrantReadWriteLock</code>	Read-heavy shared state	Concurrent readers, exclusive writer
<code>StampedLock</code>	Optimised read-heavy access	Optimistic reads with validation
<code>Semaphore</code>	Limit concurrency	Configurable number of permits
<code>Condition</code>	Wait for state changes	Multiple wait sets per explicit lock

Final Takeaway

Advanced locking is not about replacing `synchronized` everywhere.

Each tool expresses a different coordination rule:

```

ReentrantLock
    → Exactly one owner, with flexible acquisition

ReadWriteLock
    → Many readers or one writer

StampedLock
    → Read optimistically, then validate

Semaphore
    → Allow only a fixed number of concurrent users
  
```

Condition

→ Wait until protected state satisfies a predicate

Choose the simplest mechanism that accurately represents the shared-state rule. More advanced locks provide additional control, but they also increase the responsibility placed on the programmer.